IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2026, No. 4, pp. 890–916. DOI:10.46586/tches.v2026.i4.890-916

From Multiplication to Recovery: A Single-trace Side-Channel Attack on HQC

Yikang Guo[1*], Yuchang Hu[1*], Yanbin Li[1(✉)], Shilin Sun[1], Geng Chen[1], Chunpeng Ge[1] and Jiankuo Dong[2]

[1] School of Software, Shandong University, Jinan, China,
yanbinli@sdu.edu.cn,Guoyikang0910@163.com
[2] School of Cyber Science and Technology, University of Science and Technology of China, Hefei, China

Abstract. Hamming Quasi-Cyclic (HQC) has been selected by NIST as a standard for code-based Post-Quantum Cryptography (PQC) Key Encapsulation Mechanisms (KEMs). While HQC offers robust theoretical security based on the Decisional Quasi-Cyclic Syndrome Decoding problem, its physical implementation is vulnerable to side-channel attacks. Previous research has predominantly focused on vulnerabilities during the decoding stage. However, the Karatsuba polynomial multiplication in the latest HQC specifications has not been thoroughly analyzed. In this work, we present a comprehensive Soft-Analytical Side-Channel Attack (SASCA) targeting the implementation of the iterative Karatsuba algorithm. We present the optimized factor graph and algorithm tailored to the binary values and sparsity of HQC to make the SASCA more practical. Furthermore, we introduce a joint recovery framework that combines the leakage of polynomial multiplication with the leakage of decoder using valid ciphertexts. We demonstrate the efficiency of our attack against the reference implementation and extend our attack to the Liboqs library, achieving a high success rate even in the presence of high noise levels.

Keywords: Post-Quantum Cryptography · Hamming Quasi-Cyclic (HQC) · Karatsuba · Side-channel Attack

1 Introduction

The rapid advancement of quantum computing technology poses a serious threat to classical public-key cryptographic primitives, particularly those relying on the hardness of integer factorization and the discrete logarithm problem. To mitigate this risk, the National Institute of Standards and Technology (NIST) initiated a standardization process for Post-Quantum Cryptography (PQC) in 2016. While the initial selections for standardization focused on lattice-based schemes (ML-KEM, ML-DSA), NIST initiated the fourth round to increase the diversity of hardness assumptions.

Hamming Quasi-Cyclic (HQC) has recently been selected as a new PQC key encapsulation mechanism (KEM) standard. Although HQC is secure at the algorithm level, its implementations have been shown to be vulnerable to side-channel attacks.

The initial versions of HQC that used BCH codes had been targeted by timing attacks [WTBB+19, PT19] and power attacks [SRSWZ20]. The scheme had also been successfully analyzed by a decryption failure attack [GJ20]. A new version based on concatenated Reed-Muller (RM) and Reed-Solomon (RS) codes was proposed during the third round of the NIST standardization. The leakage of the Reed-Solomon outer code was exploited

*These authors contributed equally to this work.

Licensed under Creative Commons License CC-BY 4.0.
Received: 2026-04-15 Accepted: 2026-07-15 Published: 2026-09-11

to construct chosen ciphertext attacks [SHR+22]. Several attacks exploited the leakages of Reed-Muller inner code [BMG+25, DG25, PRJ+25, DGN25, MNMD25]. Most of them exploited the leakage of `expand_and_sum` operation during processing the input of Reed-Muller decoding bit by bit [BMG+25, PRJ+25, DGN25, MNMD25]. Combining the `expand_and_sum` leakage and post-processing formed efficient attack methods for recovering the secret key [PRJ+25, DGN25].

1.1 Contributions

This work investigates the most recent implementation of HQC, released on August 22, 2025, and analyzes the iterative Karatsuba algorithm used for polynomial multiplication. We show how to construct the factor graphs in the iterative Karatsuba algorithm, more importantly, optimize the factor graphs and algorithms to make the soft-analytical side-channel attack (SASCA) more practical. We also extend the attack to other open-source HQC variants, such as the Liboqs library, providing optimized factor graph models and algorithms. Furthermore, by exploiting the critical `expand_and_sum` leakage point, we introduce a joint recovery framework to retrieve the secret key by combining the leakage of polynomial multiplication and decoder using legitimate ciphertexts.

First, we analyze the characteristics and vulnerabilities of Karatsuba implementation in HQC. We exhibit the new leakage points during the polynomial multiplication. Second, we show how to construct the factor graphs in the Karatsuba algorithm. To make the SASCA more practical, we present the optimized factor graph and algorithm according to the binary values and sparsity of HQC. Finally, we introduce a novel attack by combining the Karatsuba leakage with `expand_and_sum`, which was considered as the most threatening leakage point under valid ciphertexts. The source code is available at Github[1].

1.2 Comparison with Related Works

Among the NIST PQC candidates, NTT-based polynomial multiplication is used in the Kyber [BDK+18] and Dilithium [DKL+18], while Saber [DKSRV18], NTRU-Prime [BCLVV18], and HQC [MAB+18] use Toom-Cook-based [Too63] and Karatsuba-based polynomial multiplications [KO62]. There were many works to perform SASCA on NTT-based polynomial multiplication [PPM17, PP19, HHP+21]. The vulnerabilities of Toom-Cook-based polynomial multiplication were also analyzed and exploited to perform single-trace attacks [LZH+22]. Although both [LZH+22] and our work apply SASCA to divide-and-conquer polynomial multiplications, the computational structures of Karatsuba and Toom-Cook are different. Karatsuba relies on recursive two-way splitting, whereas Toom-Cook utilizes multi-point evaluation and interpolation. Consequently, they exhibit distinct leakage characteristics, variable dependencies, and factor graph structure. Rabas et al. presented a machine learning attack on the multiplication of the additional implementation in the February 2025 version [RBG+25]. Recently, Lai et al. found timing leakage in the latest HQC implementation and recovered 94% of the secret key bits [LZZ+25]. So far, there have been no power attacks targeting the iterative Karatsuba polynomial multiplication in the latest version of HQC.

SASCA was proposed to jointly exploit the algorithmic dependencies across multiple intermediate variables using factor graphs [VCGS14], which has recently emerged as a significant threat to HQC. Goy et al. presented the factor graph for the RS decoder of HQC to recover the ephemeral shared key [GMGL24]. During the RM decoder, the factor graph of the Fast Hadamard Transform (FHT) function was built to recover the shared key [BMG+25]. It was also extended to recover the long-term secret key using chosen invalid ciphertexts [BMG+25]. However, attacks with malicious ciphertexts will fail against the countermeasure of detection of an invalid ciphertext [RCDB24, XPR+21].

[1] https://github.com/DawnGlow/Single-trace-on-HQC.git

Table 1: Comparison of power attacks on HQC-128

Work	Traces	Target	Method	Ciphertext
Ours	1	Karatsuba	SASCA	Valid
[DGN25]	1	`expand_and_sum`	MDPC	Valid
[PRJ+25]	2	`expand_and_sum`	Likelihood	Valid
[MNMD25]	83	`expand_and_sum`	Linear Algebra	Valid
[SHR+22]	52992	RM decoder output	Zero Tester	Valid
[BMG+25]	1	FHT	SASCA	Invalid

To carry out the attacks for long-term secret key recovery that rely on valid ciphertexts, Paiva et al. presented the leakage of the `expand_and_sum` operation and introduced a likelihood solver for post-processing [PRJ+25]. The `expand_and_sum` leakage $c' = v - u \cdot y$ was also exploited to recover the secret key y by Maillet et al.'s algebraic approach [MNMD25] and Dong et al.'s moderate-density parity-check (MDPC) decoding for propagating the side-channel probabilities of c' to x, y [DGN25]. In this work, we discuss how to combine the leakage of polynomial multiplication and `expand_and_sum` under the valid ciphertext to construct a more efficient single-trace attack. We propose a joint recovery framework to retrieve the secret key using legitimate ciphertexts, achieving low error bits at high noise levels. Table 1 presents a comparison between our proposed method and related works.

2 Preliminaries

2.1 HQC

Hamming Quasi-Cyclic (HQC) [GAMA+25] is a code-based key encapsulation mechanism. The IND-CPA secure public-key encryption scheme HQC.CPAPKE consists of three core algorithms: Key Generation, Encryption, and Decryption. All operations are performed in the ring $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, where n is a prime number. The primary characteristic of HQC is the application of a public binary linear code $\mathcal{C}$ for error correction, which is constructed by cascading a Reed-Solomon code with a repeated Reed-Muller code. During encoding, the message m is encoded using an externally shortened Reed-Solomon code into n_1 bytes. Each byte is then encoded into n_2 bits using the internally repeated Reed-Muller code. Finally, the n_1 binary output blocks from the Reed-Muller code are concatenated to form a codeword of length $n = n_1 n_2$. The decoding process reverses this procedure.

The key generation algorithm randomly samples a polynomial $h \leftarrow \mathcal{R}$, and two sparse polynomials $x, y \leftarrow (\mathcal{R}, \omega)$, where the Hamming weight $w_H(x) = w_H(y) = \omega$. The secret key is $sk = (x, y)$, and the public key is $pk = (h, s)$, where $s = x + h \cdot y$. The encryption algorithm deterministically generates three error vectors from a random seed θ: two vectors $r_1, r_2 \in \mathcal{R}$ with Hamming weight ω_r, and an error vector $e \in \mathcal{R}$ with Hamming weight ω_e. The message $m \in \{0,1\}^k$ is encoded into the codeword $m\mathbf{G}$ using the encoding function Encode, where $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is the generator matrix of the linear code $\mathcal{C}$. The ciphertext (u, v) is then computed as

$$u = r_1 + h \cdot r_2, \quad v = m\mathbf{G} + s \cdot r_2 + e.$$

The decryption algorithm is used to recover the m from ciphertext (u, v). When receiving the ciphertext (u, v), the algorithm computes

$$c' = v - u \cdot y = m\mathbf{G} + (s \cdot r_2 + e) - (r_1 + h \cdot r_2) \cdot y = m\mathbf{G} + (x \cdot r_2 - r_1 \cdot y + e).$$

where the effective error vector is $e' = x \cdot r_2 - r_1 \cdot y + e$. Since x, y, r_1, r_2 and e all have low Hamming weight, appropriate parameter choices ensure that the Hamming weight

$w_H(e')$ of e' remains low. As long as it does not exceed the error-correction capacity of the code $\mathcal{C}$, the original message m can be successfully recovered by the concatenated decoder.

2.2 Karatsuba

Given two polynomials $A(x)$ and $B(x)$ to be multiplied, the polynomial $A(x)$ can be split into two $n/2$-degree polynomials $a_h(x)$ and $a_l(x)$, where $A(x) = a_h(x) \cdot x^{n/2} + a_l(x)$.

$$\begin{aligned} a_h(x) &= a_{h,n/2-1} \cdot x^{n/2-1} + \cdots + a_{h,1} \cdot x + a_{h,0}, \\ a_l(x) &= a_{l,n/2-1} \cdot x^{n/2-1} + \cdots + a_{l,1} \cdot x + a_{l,0}. \end{aligned} \tag{1}$$

Similarly, the polynomial $B(x)$ can also be split into two $n/2$-degree polynomials $b_h(x)$ and $b_l(x)$, where $B(x) = b_h(x) \cdot x^{n/2} + b_l(x)$. In this case, traditional multiplication requires computing the four sub-products: $a_h(x) \cdot b_h(x)$, $a_h(x) \cdot b_l(x)$, $a_l(x) \cdot b_h(x)$, and $a_l(x) \cdot b_l(x)$. However, Karatsuba multiplication uses algebraic transformations to reduce the multiplication to

$$\begin{aligned} C(x) &= A(x) \cdot B(x) \\ &= (a_h(x) \cdot x^{n/2} + a_l(x)) \cdot (b_h(x) \cdot x^{n/2} + b_l(x)) \\ &= a_h(x) \cdot b_h(x) \cdot x^n + (a_h(x) \cdot b_l(x) + a_l(x) \cdot b_h(x)) \cdot x^{n/2} + a_l(x) \cdot b_l(x) \\ &= a_h(x) \cdot b_h(x) \cdot x^n + \big[(a_h(x) + a_l(x)) \cdot (b_h(x) + b_l(x)) - a_h(x) \cdot b_h(x) \\ &\quad - a_l(x) \cdot b_l(x)\big] \cdot x^{n/2} + a_l(x) \cdot b_l(x). \end{aligned} \tag{2}$$

There are only three sub-products that need to be computed: $a_h(x) \cdot b_h(x)$, $(a_h(x) + a_l(x)) \cdot (b_h(x) + b_l(x))$, and $a_l(x) \cdot b_l(x)$, with the rest of the operations being small-cost additions and subtractions. Thus, Karatsuba multiplication reduces the time complexity to $O(n^{\log_2 3})$. For sub-multiplications that are still large after splitting, Karatsuba can be recursively executed until they are reduced to a sufficiently small size.

2.3 Soft-Analytical Side-Channel Attacks

Soft-Analytical Side-Channel Attacks (SASCA) is a powerful attack method that combines side-channel information leaked from the implementation of cryptographic algorithms with the algebraic structure of the algorithm. The adversary models the target cryptographic algorithm as a factor graph including the variable nodes and factor nodes. The variable nodes represent intermediate variables of the algorithm, while the factor nodes represent the constraints between these variables. Integrating the side-channel probability information into the factor graph, the adversary can execute the Belief Propagation (BP) algorithm to determine the marginal probability distribution of the sensitive information. The classic BP algorithm is the sum-product BP in the probability domain, where messages are updated by performing sum-product operations:

Message from factor nodes to variable nodes. For a factor node f_j, which is connected to a set of variable nodes $\{x_k\}$, let $\mathcal{N}(f_j)$ represent the set of all variable nodes adjacent to f_j. $\mathbf{x}_j$ denotes the vector of all variable nodes connected to f_j, and $\psi_j(\mathbf{x}_j)$ represents the potential function (i.e., constraint relationship) or conditional probability of the factor node. The message update rule is

$$m_{f_j \to x_i}(x_i) = \sum_{\mathbf{x}_j \setminus x_i} \psi_j(\mathbf{x}_j) \prod_{x_k \in \mathcal{N}(f_j) \setminus \{x_i\}} m_{x_k \to f_j}(x_k) \tag{3}$$

Message from variable nodes to factor nodes. Let a variable node x_i be adjacent to a set of factor nodes f_k, where $\mathcal{N}(x_i)$ represents the set of all factor nodes adjacent to

x_i. The message update rule is given by

$$m_{x_i \to f_j}(x_i) = \prod_{f_k \in \mathcal{N}(x_i) \setminus \{f_j\}} m_{f_k \to x_i}(x_i) \tag{4}$$

3 Vulnerabilities Analysis of Recursive Karatsuba

In this section, we analyze the leakage characteristics of polynomial multiplication in HQC decryption. First, we describe the Karatsuba implementation used in HQC polynomial multiplication. Then, we reveal the side-channel vulnerabilities within this process.

To perform the polynomial multiplication, HQC recursively applies the Karatsuba algorithm to decompose the large-scale multiplication into smaller ones, ultimately using the basic Schoolbook multiplication as the recursive base when the degree is reduced to 16. This structure strikes a balance between recursion depth and multiplication complexity. Algorithm 1 shows the pseudocode for the Karatsuba multiplication.

Algorithm 1 Karatsuba Multiplication

1: **Input:** a, b are two polynomials of degree n
2: **Output:** r is the product of a and b
3: **function** KARATSUBA_MUL(r, a, b, n)
4: **if** $n \leq 16$ **then**
5: SCHOOLBOOK_MUL(r, a, b, n)
6: **return**
7: **end if**
8: $m \leftarrow \lfloor n/2 \rfloor$
9: $n_{\text{low}} \leftarrow m$, $n_{\text{high}} \leftarrow n - m$
10: $Z_0 \leftarrow$ KARATSUBA_MUL($z_0, a_{\text{low}}, b_{\text{low}}, n_{\text{low}}$)
11: $Z_2 \leftarrow$ KARATSUBA_MUL($z_2, a_{\text{high}}, b_{\text{high}}, n_{\text{high}}$)
12: $a_{\text{mid}} \leftarrow a_{\text{low}} \oplus a_{\text{high}}$
13: $b_{\text{mid}} \leftarrow b_{\text{low}} \oplus b_{\text{high}}$
14: $Z_1 \leftarrow$ KARATSUBA_MUL($z_1, a_{\text{mid}}, b_{\text{mid}}, n_{\text{high}}$)
15: $r[0 : 2n_{\text{low}} - 1] \leftarrow r[0 : 2n_{\text{low}} - 1] \oplus Z_0$
16: $r[2n_{\text{low}} : 2n - 1] \leftarrow r[2n_{\text{low}} : 2n - 1] \oplus Z_2$
17: $r[n_{\text{low}} : n_{\text{low}} + 2n_{\text{high}} - 1] \leftarrow r[n_{\text{low}} : n_{\text{low}} + 2n_{\text{high}} - 1] \oplus (Z_1 \oplus Z_0 \oplus Z_2)$
18: **end function**

During the decryption operation $c' = v - u \cdot y$ in HQC, the recursive Karatsuba multiplication is used to compute $u \cdot y$, where the ciphertext u and the secret key y are both polynomials of degree $n = 277$. The algorithm recursively splits u and y until $n \leq 16$, at which point Schoolbook multiplication is executed. At each recursion, y is split into three subsets: y_j^i, y_{j+1}^i, and y_{j+2}^i, where i denotes the i-th recursion round, y_j^i is the lower $\lfloor n/2 \rfloor$ polynomial coefficients of y, y_{j+2}^i is the higher $\lceil n/2 \rceil$ polynomial coefficients of y, and y_{j+1}^i is the result of $y_j^i + y_{j+2}^i$ (denoted as XOR in the algorithm). Similarly, at each recursion, u is split into u_j^i, u_{j+1}^i, and u_{j+2}^i.

Algorithm 2 shows the pseudocode of Schoolbook multiplication based on the HQC reference implementation. In this algorithm, a is a subset of the secret key y, specifically y_j^4, and b is a subset of the ciphertext u, specifically u_j^4. The superscript 4 indicates that the Schoolbook multiplication is executed at the 4-th recursion round. In Line 7 of Algorithm 2, the *bit*-th bit of $a[i]$ is extracted, and a 64-bit integer *mask* is generated based on this value. If the *bit*-th bit of $a[i]$ is 0, the *mask* is set to all zeros. If the *bit*-th bit of $a[i]$ is 1, the *mask* is set to all ones. Due to the significant Hamming weight difference between all-zeros and all-ones, this step introduces a distinct power consumption difference.

Moreover, in Lines 9 and 10 of Algorithm 2, the *mask* performs a bitwise AND operation with the ciphertext subset b, and the result is XORed with the array r. Since the *mask* consists of either all zeros or all ones, the result of this AND operation can only be 0 or $b[j]$. We observe bit flips in r via power side-channel analysis. If r flips, it implies that the *mask* is all-ones. If r does not flip, then either the *mask* is all-zeros or $b[j]$ is 0. However, since $b[j]$ is a 64-bit word, the probability of it being 0 is $1/2^{64}$, which can be ignored. Therefore, if no flip occurs in r, the *mask* is deduced to be all-zeros. Since the Schoolbook multiplication processes each bit of a, we can recover all positions of a through side-channel attacks.

Algorithm 2 Schoolbook Multiplication with Leakage Points

1: **Input:** Secret key subset a and ciphertext subset b (arrays of n 64-bit words)
2: **Output:** Resulting product $r = a \times b$
3: **function** SCHOOLBOOK_MUL(r, a, b, n)
4: $r \leftarrow \mathbf{0}$
5: **for** $i = 0$ **to** $n - 1$ **do**
6: **for** $bit = 0$ **to** 63 **do**
7: $mask \leftarrow -((a[i] \gg bit) \wedge 1)$ ▷ Leakage of y
8: **for** $j = 0$ **to** $n - 1$ **do**
9: $r[i + j] \leftarrow r[i + j] \oplus ((b[j] \ll bit) \wedge mask)$ ▷ Leakage of r
10: $r[i + j + 1] \leftarrow r[i + j + 1] \oplus ((b[j] \gg (64 - bit)) \wedge mask)$ ▷ Leakage of r
11: **end for**
12: **end for**
13: **end for**
14: **end function**

As Schoolbook multiplication sequentially performs $y_j^4 \cdot u_j^4$, $y_{j+2}^4 \cdot u_{j+2}^4$, and $y_{j+1}^4 \cdot u_{j+1}^4$, we can use side-channel attacks to recover y_j^4, y_{j+1}^4, and y_{j+2}^4. Then, based on the property that $y_{j+1}^4 = y_j^4 + y_{j+2}^4$, we use the SASCA method to verify the correctness of classification.

4 SASCA on Recursive Karatsuba

While Template Attacks (TA) can extract information from highly leaky operations, they perform independent classifications for each target variable. In the single-trace scenario, even prominent leakages are inevitably interfered by localized noise, resulting in the errors that TA cannot resolve. To overcome this limitation and maximize the utilization of available leakages, SASCA is introduced to exploit the deterministic mathematical relationships. In this section, we describe how SASCA can be applied to the recursive Karatsuba algorithm in HQC. We first construct a factor graph to model the relationships of intermediate variables in Karatsuba, and then optimize the BP algorithm to enhance the efficiency of SASCA.

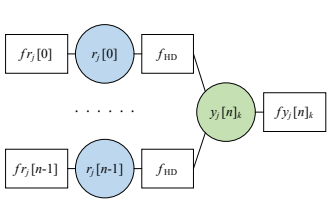
4.1 Factor Graphs of Recursive Karatsuba

As described in Section 3, the Schoolbook multiplication in the HQC reference implementation contains two leakage points.

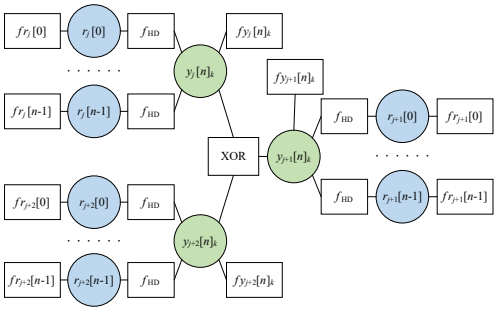
First, the bitwise extraction process in Line 7 of Algorithm 2 leaks information about each bit of the secret key y_j^4. For simplicity, we omit the recursion round and denote it as y_j. For the input n-degree polynomials, where $n = 8$ or 9, we represent the k-th bit of the secret key as $y_j[n]_k$. Second, in Lines 9 and 10 of Algorithm 2, the bitwise AND operation between the secret key bit $y_j[n]_k$ and the ciphertext u_j is performed, and the result is XOR with the array r. Since whether a bit in r flips is directly related to the value of $y_j[n]_k$,

we record whether the bits in r flip each time the XOR operation is executed through side-channel observation, denoted as $r_j[n]$. Here, n represents the polynomial degree of the input to the Schoolbook multiplication, and also the number of XOR operations on r. The factor graph for the Schoolbook multiplication is shown in Figure 1a. In the factor graph, the dark gray variable nodes correspond to the leakage of y in Line 7 of Algorithm 2, while the light gray variable nodes correspond to the leakage of r in Lines 9 and 10 of Algorithm 2.

In the first leakage point, the leaked information is a single-bit value $y_j[n]_k$ of the secret key, so it is a binary classification problem. The marginal probability distribution of $y_j[n]_k$, denoted as $fy_j[n]_k$, follows a binomial distribution. In the second leakage point, it is not necessary to precisely recover the value of r, but only to observe whether bits flip in r. This can also be modeled as a binary classification problem. The marginal probability distribution of $r_j[n]$, denoted as $fr_j[n]$, also follows a binomial distribution. Assume that a bit flip exists in r, in which case $r_j[n]$ is classified as 1, and no flip as 0. Under this assumption, a factor node f_{HD} satisfies the constraint $y_j[n]_k = r_j[n]$, which is equivalent to $y_j[n]_k \oplus r_j[n] = 0$. Therefore, the factor graph for the Schoolbook multiplication only contains binomial probability distributions that satisfy the parity-check constraint.



(a) Factor graph of Schoolbook multiplication.



(b) Factor graph of Karatsuba multiplication.

Figure 1: Factor graphs of the reference implementation.

In the Karatsuba multiplication, the Schoolbook multiplication is called to sequentially perform $y_j \cdot u_j$, $y_{j+2} \cdot u_{j+2}$, and $y_{j+1} \cdot u_{j+1}$. Based on the relationship $y_{j+1} = y_j \oplus y_{j+2}$, it can be observed that each corresponding bit in these terms also has the relationship $y_{j+1}[n]_k = y_j[n]_k \oplus y_{j+2}[n]_k$. We use the factor graph to model this relationship. The factor graph for the Karatsuba multiplication is shown in Figure 1b.

A factor graph for Karatsuba multiplication consists of three Schoolbook multiplication factor graphs, which are connected through XOR operations. By performing BP on the Karatsuba factor graph, we can recover 2 bits of the secret key $y_j[n]_k$ and $y_{j+2}[n]_k$. We perform BP on the same factor graph for each bit of the secret key y, ultimately recovering all the bits. The relationship $y_{j+1}[n]_k = y_j[n]_k \oplus y_{j+2}[n]_k$ can be expressed as $y_j[n]_k \oplus y_{j+1}[n]_k \oplus y_{j+2}[n]_k = 0$, the factor graph for the Karatsuba multiplication also only contains binomial probability distributions that satisfy the parity-check constraint. Based on this property, BP on the Karatsuba multiplication factor graph can be optimized.

4.2 Improving Practical SASCA

The BP algorithm is used to update the marginal probability distributions in the factor graph. The sum-product BP algorithm in the probability domain guarantees precise convergence to the true marginal distribution. However, the efficiency of repeated execution

on the factor graph of Karatsuba multiplication is limited. Therefore, we optimize the BP strategy to improve the execution efficiency.

4.2.1 Belief Propagation in the Log-Likelihood Ratio Domain

The Log-Likelihood Ratio (LLR) was originally proposed in information theory for the efficient decoding of error-correcting codes [Gal03], which was also used for the initialization of BP [TO10, DGN25]. Compared to the sum-product algorithm that relies on the continuous multiplication of probabilities, LLR maps probabilities to the logarithmic domain, transforming computationally expensive probability multiplications into highly efficient and numerically stable additions. In the factor graph of Karatsuba multiplication, all variable nodes are binary variables. Using LLR instead of probability distributions for message passing can significantly improve computational efficiency. For any binary variable x_i in the factor graph, its LLR is defined as

$$L(x_i) = \ln \frac{P(x_i = 0)}{P(x_i = 1)} \tag{5}$$

Correspondingly, the probability distributions $P(x_i = 0)$ and $P(x_i = 1)$ can be expressed in terms of the LLR as

$$P(x_i = 0) = \frac{e^{L(x_i)}}{1 + e^{L(x_i)}}, \quad P(x_i = 1) = \frac{1}{1 + e^{L(x_i)}} \tag{6}$$

Theorem 1. *When the factor graph contains only binary probability distributions and satisfies the parity-check constraint, the probability distribution by addition and hyperbolic tangent operations in the Log-Likelihood Ratio domain BP (LLR-BP) is equivalent to the probability distribution by multiplication in the probability domain BP.*

Proof. For binomial probability distributions, we introduce the probability difference $y_i = P(x_i = 0) - P(x_i = 1)$ as a key variable in the message passing. The mathematical relationship between the probability difference and LLR can be simplified as

$$y_i = P(x_i = 0) - P(x_i = 1) = \frac{e^{L(x_i)} - 1}{e^{L(x_i)} + 1} = \tanh\left(\frac{L(x_i)}{2}\right) \tag{7}$$

Message from factor node to variable node. When the constraints in the factor graph satisfy parity-check constraints, the potential function for the factor node is $\psi_j(\mathbf{x}_j) = \mathbb{I}\left(\bigoplus_{x_k \in \mathcal{N}(f_j)} x_k = 0\right)$, where $\bigoplus$ denotes modulo-2 addition, and $\mathbb{I}$ denotes indicator function. This constraint requires the XOR of all adjacent variables to be zero. To facilitate message passing, we need to transform the modulo-2 addition into a form suitable for the sign domain. Introducing the mapping $h_k = (-1)^{x_k}$, the modulo-2 addition operations can be equivalently represented by a product relation as

$$\bigoplus_{k \neq i} x_k = 0 \iff \prod_{k \neq i} h_k = 1; \qquad \bigoplus_{k \neq i} x_k = 1 \iff \prod_{k \neq i} h_k = -1 \tag{8}$$

Consequently, we can obtain where $h_k = (-1)^{x_k}$, we have

$$m_{f_j \to x_i}(0) - m_{f_j \to x_i}(1) = P\left(\prod_{k \neq i} h_k = 1\right) - P\left(\prod_{k \neq i} h_k = -1\right) = \prod_{k \neq i} y_k, \tag{9}$$

where $y_k = P(h_k = 1) - P(h_k = -1) = P(x_k = 0) - P(x_k = 1)$.

Since $m_{f_j \to x_i}(0) + m_{f_j \to x_i}(1) = 1$, we can solve for the following equations by combining with Equation (9):

$$m_{f_j \to x_i}(0) = \frac{1}{2} + \frac{1}{2} \prod_{x_k \in \mathcal{N}(f_j) \setminus \{x_i\}} y_k, \quad m_{f_j \to x_i}(1) = \frac{1}{2} - \frac{1}{2} \prod_{x_k \in \mathcal{N}(f_j) \setminus \{x_i\}} y_k \tag{10}$$

Equation (10) shows the message update formula from the factor node to the variable node. By dividing the two expressions in Equation (10) and taking the logarithm, we get

$$L_{f_j \to x_i} = \ln \frac{m_{f_j \to x_i}(0)}{m_{f_j \to x_i}(1)} = \ln \left(\frac{1 + \prod_{k \neq i} y_k}{1 - \prod_{k \neq i} y_k} \right) = 2 \cdot \operatorname{atanh} \left(\prod_{x_k \in \mathcal{N}(f_j) \setminus \{x_i\}} y_k \right) \tag{11}$$

Finally, by substituting $y_k = \tanh\left(L_{x_k \to f_j}/2\right)$, we obtain the message update formula from the factor node to the variable node in the LLR domain as

$$L_{f_j \to x_i} = 2 \cdot \operatorname{atanh} \left(\prod_{x_k \in \mathcal{N}(f_j) \setminus \{x_i\}} \tanh \left(\frac{L_{x_k \to f_j}}{2} \right) \right) \tag{12}$$

Message from variable node to factor node. For the message update from variable nodes, the logarithmic properties of the LLR can transform the product in the probability domain into a summation form. The LLR-BP derivation for the message from the variable node to the factor node is as

$$L_{x_i \to f_j} = \ln \frac{m_{x_i \to f_j}(0)}{m_{x_i \to f_j}(1)} = \sum_{f_k \in \mathcal{N}(x_i) \setminus \{f_j\}} \ln \frac{m_{f_k \to x_i}(0)}{m_{f_k \to x_i}(1)} = \sum_{f_k \in \mathcal{N}(x_i) \setminus \{f_j\}} L_{f_k \to x_i} \tag{13}$$

Considering the initial prior information of the variable node, L_{init}, the complete variable node message update is

$$L_{x_i \to f_j} = L_{\text{init}} + \sum_{f_k \in \mathcal{N}(x_i) \setminus \{f_j\}} L_{f_k \to x_i} \tag{14}$$

LLR-BP transforms the message representation in the probability domain, where multiplication is replaced by addition and the hyperbolic tangent operation in the logarithmic domain. This transformation is purely a mathematical form conversion and does not alter the underlying information passing structure or the iterative convergence of the algorithm. Therefore, both representations are mathematically equivalent, and the resulting posterior probability distribution is consistent. □

Time complexity analysis. Compared to the probability domain BP algorithm, the advantage of LLR-BP mainly lies in the reduction of computational complexity. For the probability domain BP algorithm, suppose there is a factor $\psi(\mathbf{x})$ with d neighboring variables. The computation of the message $m_{f \to x_i}(x_i)$ requires traversing all 2^{d-1} possible combinations of values for the $d-1$ other variables, and for each combination, the message product and factor function value must be computed, leading to an exponential time complexity in the degree d of the factor node, which is $O(2^d)$. In contrast, when using LLR-BP with parity-check constraints, the message update can be converted into a linear closed-form solution through LLR representation. It only requires computing the product of $d-1$ terms and applying a single atanh transformation, thereby reducing the total time complexity to $O(d)$.

4.2.2 Min-Sum Based Log-Likelihood Ratio Belief Propagation

Although LLR-BP effectively reduces the theoretical complexity of the algorithm, the hyperbolic tangent operation still introduces a significant computational overhead in practical execution. The min-sum algorithm is a simplified approximation of the LLR-BP algorithm, designed to further reduce the complexity of factor node updates. By approximating the complex hyperbolic functions in LLR-BP, the transcendental function calculations are replaced with simple comparisons and multiplications, thereby significantly reducing the time complexity of implementation.

Since $\tanh(x)$ is an odd function and $|x|$ is monotonically increasing when $x \geq 0$, the messages can be decomposed into two parts: the sign and the magnitude. First, each variable node message can be expressed as

$$L_{x_k \to f_j}^{(t-1)} = \operatorname{sgn}\left(L_{x_k \to f_j}^{(t-1)}\right) \cdot \left|L_{x_k \to f_j}^{(t-1)}\right| \tag{15}$$

where t denotes the iteration number of the BP algorithm, and $\operatorname{sgn}(\cdot)$ is the sign function, which extracts the sign information of the message. Substituting Equation (15) into Equation (12) yields

$$L_{f_j \to x_i}^{(t)} = 2 \cdot \operatorname{atanh}\left(\prod_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \operatorname{sgn}\left(L_{x_k \to f_j}^{(t-1)}\right) \cdot \tanh\left(\frac{\left|L_{x_k \to f_j}^{(t-1)}\right|}{2}\right)\right) \tag{16}$$

Similarly, using the odd function property of the $\operatorname{atanh}(x)$ function, the sign product can be extracted outside the $\operatorname{atanh}(x)$ function

$$L_{f_j \to x_i}^{(t)} = 2 \cdot \prod_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \operatorname{sgn}\left(L_{x_k \to f_j}^{(t-1)}\right) \cdot \operatorname{atanh}\left(\prod_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \tanh\left(\frac{\left|L_{x_k \to f_j}^{(t-1)}\right|}{2}\right)\right) \tag{17}$$

At this point, the magnitude part in Equation (17) still contains complex hyperbolic function operations. The core approximation of the min-sum algorithm is to approximate the product of multiple $\tanh(x)$ functions by the $\tanh(x)$ function corresponding to the minimum value among them.

$$\begin{aligned} \prod_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \tanh\left(\frac{\left|L_{x_k \to f_j}^{(t-1)}\right|}{2}\right) &\approx \min_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \left(\tanh\left(\frac{\left|L_{x_k \to f_j}^{(t-1)}\right|}{2}\right)\right) \\ &= \tanh\left(\frac{1}{2} \min_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \left|L_{x_k \to f_j}^{(t-1)}\right|\right) \end{aligned} \tag{18}$$

Substituting Equation (18) into Equation (17), we obtain

$$L_{f_j \to x_i}^{(t)} \approx \prod_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \operatorname{sgn}\left(L_{x_k \to f_j}^{(t-1)}\right) \cdot \min_{x_k \in \mathcal{N}(f_j)\backslash\{x_i\}} \left|L_{x_k \to f_j}^{(t-1)}\right| \tag{19}$$

Equation (19) is the factor node update equation for the min-sum algorithm. Compared to LLR-BP, this equation avoids complex tanh and atanh function calculations, requiring only sign multiplication and minimum value searching. While the time complexity of the min-sum algorithm remains $O(d)$ as in LLR-BP, the actual computational complexity is significantly reduced.

However, the simplification in the min-sum algorithm is an approximation, so the computed factor node message values may have errors compared to the exact values, particularly overestimating the magnitude. This slows down convergence and reduces

accuracy. To compensate for this approximation error, we use the normalized min-sum algorithm. The main idea behind the normalized min-sum algorithm is to multiply the result of the min-sum algorithm by a correction factor α to reduce the discrepancy between the approximate and exact values.

$$L_{f_j \to x_i}^{(t)} = \alpha \cdot \prod_{x_k \in \mathcal{N}(f_j) \setminus \{x_i\}} \operatorname{sgn}\left(L_{x_k \to f_j}^{(t-1)}\right) \cdot \min_{x_k \in \mathcal{N}(f_j) \setminus \{x_i\}} \left|L_{x_k \to f_j}^{(t-1)}\right| \tag{20}$$

where α is the normalization factor, with $0 < \alpha < 1$.

Theorem 2. *The hard decision of the normalized min-sum algorithm is equal to the hard decision of the sum-product algorithm.*

Proof. **Error in single message passing.** Let the degree of the factor node be d, and the input message magnitudes be $a_k = |L_{x_k \to f_j}| \geq 0$. The magnitude computation in the sum-product (SP) algorithm is given by

$$A_{\mathrm{SP}} = 2 \cdot \operatorname{atanh}\left(\prod_{k=1}^{d-1} \tanh\left(\frac{a_k}{2}\right)\right) \tag{21}$$

Let $m = \min_k a_k$, and $t_{\min} = \tanh(m/2)$. Since $\tanh(a_k/2) \leq 1$, we have

$$\prod_{k=1}^{d-1} \tanh\left(\frac{a_k}{2}\right) \leq t_{\min} \tag{22}$$

When other a_k are much larger than m, the product approaches $t_{\min}$. Therefore, the magnitude in SP satisfies $A_{\mathrm{SP}} \leq 2 \cdot \operatorname{atanh}(t_{\min}) = m$. The minimum value m is an upper bound for the SP magnitude. The normalized min-sum (NMS) algorithm approximates A_{SP} with $\alpha \cdot m$. Since SP is usually less than m, choosing $0 < \alpha < 1$ can reduce the error. For any $\epsilon > 0$, there exists $\alpha \in (0, 1)$ such that $|\alpha \cdot m - A_{\mathrm{SP}}| < \epsilon$.

Error control in the iterative process. Let the SP and NMS message vectors in the t-th iteration be $\mathbf{L}_{\mathrm{SP}}^{(t)}$ and $\mathbf{L}_{\alpha}^{(t)}$, respectively. Since the factor node update error is bounded by ϵ, and the variable node update is a linear summation, the maximum message error after a single iteration satisfies

$$\|\mathbf{L}_{\mathrm{SP}}^{(t)} - \mathbf{L}_{\alpha}^{(t)}\|_{\infty} \leq d_v \cdot \epsilon \tag{23}$$

where d_v is the maximum degree of the variable nodes. In multiple iterations, the $\tanh(|x|)$ function will continuously compress the messages to the range [0, 1]. This causes the messages from previous iterations to be compressed, and thus the message error from earlier iterations will also decrease. There exists a constant $\lambda < 1$ (dependent on the factor graph structure) such that

$$\|\mathbf{L}_{\mathrm{SP}}^{(t)} - \mathbf{L}_{\alpha}^{(t)}\|_{\infty} \leq \lambda^t \cdot \|\mathbf{L}_{\mathrm{SP}}^{(0)} - \mathbf{L}_{\alpha}^{(0)}\|_{\infty} + \frac{d_v \epsilon}{1 - \lambda} \tag{24}$$

When t is large enough, the second term dominates, and the error converges to

$$\|\mathbf{L}_{\mathrm{SP}}^{(t)} - \mathbf{L}_{\alpha}^{(t)}\|_{\infty} \leq \frac{d_v \epsilon}{1 - \lambda} \tag{25}$$

Hard decision consistency. The posterior LLR for a variable node x_i is

$$L_{\mathrm{post}} = L_{\mathrm{init}} + \sum_{j \in \mathcal{N}(x_i)} L_{f_j \to x_i} \tag{26}$$

where L_{init} is the channel initial LLR. Let the degree of the variable node be d_i. The difference between the posterior LLRs of SP and NMS satisfies

$$|L_{\text{post}}^{\text{SP}} - L_{\text{post}}^{\alpha}| \leq d_i \cdot \frac{d_v \epsilon}{1-\lambda} \leq d_v^2 \cdot \frac{\epsilon}{1-\lambda} \tag{27}$$

Since ϵ can be controlled by α, by choosing an appropriate α, we can ensure that ϵ is small enough such that

$$d_v^2 \cdot \frac{\epsilon}{1-\lambda} < \min_i |L_{\text{post}}^{\text{SP}}| \tag{28}$$

At this point, for all i, we have $\text{sgn}(L_{\text{post}}^{\alpha}) = \text{sgn}(L_{\text{post}}^{\text{SP}})$, which implies that the hard decision results are consistent. □

4.3 Extend the Attack to HQC Implementation in Liboqs

Liboqs is an open-source C library developed by the Open Quantum Safe project, with the primary goal of providing a set of rigorously tested post-quantum cryptographic algorithms. The library closely follows the NIST post-quantum cryptography standardization process and has recently integrated the latest version of the HQC implementation.

The HQC implementation in Liboqs also employs recursive Karatsuba multiplication to compute $u \cdot y$. In contrast to the HQC reference implementation, the Karatsuba algorithm in Liboqs recursively splits the polynomials until the degree $n = 1$, eventually calling the base_mul function to perform the multiplication when $n = 1$. A partial C implementation of base_mul is shown in Figure 2.

```
1  static void base_mul(uint64_t *c, uint64_t a, uint64_t b) {
2    ......
3    g = 0;
4    tmp1 = a & 0x0f;
5    for (size_t i = 0; i < 16; ++i) {
6      tmp2 = tmp1 - i;
7      g ^= (u[i] & (uint64_t)(0 - (1 - ((uint64_t)(tmp2 | (0 - tmp2)) >> 63)))); }
8    l = g;
9    h = 0;
10   for (size_t i = 4; i < 64; i += 4) {
11     g = 0;
12     tmp1 = (a >> i) & 0x0f;
13     for (size_t j = 0; j < 16; ++j) {
14           tmp2 = tmp1 - j;
15           g ^= (u[j] & (uint64_t)(0 - (1 - ((uint64_t)(tmp2 | (0 - tmp2)) >> 63)))); }
16     l ^= g << i;
17     h ^= g >> (64 - i); }
18   ...... }
```

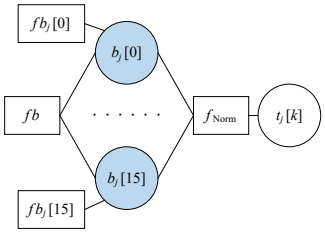
Figure 2: The C implementation of base_mul multiplication.

The input a to base_mul is a 64-bit value from the secret key y, denoted as y_j. The input b is a 64-bit value from the ciphertext u, denoted as u_j. The base_mul function splits both y_j and u_j into 16 subsets, each containing 4 consecutive bits. Each subset represents a decimal number in the range of 0 to 15. The red-highlighted portions in lines 7 and 15 of Figure 2 indicate the side-channel leakage related to each subset. Specifically, the code stores each subset of y_j in $tmp1$, which is then processed in a loop that runs 16 times. For the i-th iteration, where $0 \leq i \leq 15$, the expression $b = (0 - (1 - ((uint64_t)(tmp2|(0 - tmp2)) >> 63)))$ results in 1 when and only when $tmp1 = i$. Otherwise, b is 0. Therefore, in each iteration of the loop, only one round will result in $b = 1$, while all other rounds will yield $b = 0$. We observe the leakage at lines 7 and 15 of Figure 2 through side-channel analysis, and classify the values based on b into binary values 0 and 1. By combining the 16 b values from each loop iteration, we are able to recover each 4-bit subset of y_j.

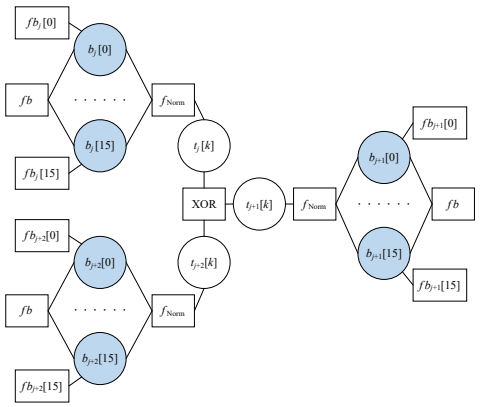
We construct factor graphs for the leakage points of Karatsuba in Liboqs. For each loop in base_mul, we observe the value of b in each iteration through side-channel analysis, denoted as $b_j[i]$, with the corresponding prior probability distribution $fb_j[i]$. To combine the $b_j[i]$ values from the 16 iterations into a 4-bit subset of y_j, denoted as $t_j[k]$ where $0 \leq k \leq 15$, we use a factor node f_{Norm} to perform weighted normalization fusion. The factor node f_{Norm} serves as an aggregator of probabilistic information, combining the probability distributions $fb_j[i]$ from the 16 iterations into the posterior probability distribution $P(t_j[k])$ for the target subset $t_j[k]$. Specifically, it extracts the probability value $P_{j,i}$ from each distribution $fb_j[i]$ where $b_j[i] = 1$, and converts these values into the likelihood of $t_j[k] = i$. Finally, by normalizing the likelihoods of all 16 $P_{j,i}$, we obtain the complete probability distribution for $t_j[k]$, expressed as

$$P(t_j[k] = i) = \frac{P_{j,i}}{\sum_{i=0}^{15} P_{j,i}}, \quad i = 0, 1, \ldots, 15. \tag{29}$$

Additionally, since only one round of $b_j[i]$ is equal to 1 in each loop, we construct a factor node f_b as a constraint based on this property. Normally, f_b would be constructed as the sum $\sum_{i=0}^{15} b_j[i] = 1$, which indicates that exactly one of the 16 values of $b_j[i]$ is equal to 1. However, we simplify the summation to a modulo-2 sum, expressed as $\bigoplus_{i=0}^{15} b_j[i] = 1$. This simplified constraint requires an odd number of 1's in $b_j[i]$, which may initially appear inconsistent with the original strict constraint. In fact, after the normalization fusion by f_{Norm}, only the position with the maximum $P_{j,i}$ will influence the final hard decision. Since the modulo-2 sum equation corrects cases where an even number of 1's occurs in $b_j[i]$ without affecting the distribution at the position of the maximum $P_{j,i}$, it does not negatively influence the final hard decision. The advantage of using this modulo-2 constraint is that it conforms to the parity-check constraint, enabling acceleration through LLR-BP as discussed in Section 4.2. The factor graph of base_mul is shown in Figure 3a. In the factor graph, the gray variable nodes correspond to the leakage of b in Lines 7 and 15 of Figure 2.



(a) Factor graph of base_mul.



(b) Factor graph of Karatsuba.

Figure 3: Factor graphs of the Liboqs implementation.

Based on the relationship $y_{j+1} = y_j \oplus y_{j+2}$, we connect the three base_mul factor graphs using XOR, forming a Karatsuba multiplication factor graph in Liboqs. Unfortunately, since each $t_j[k]$ is a 4-bit number, it is difficult to represent it using a binomial probability distribution. Therefore, we employ the sum-product algorithm in the probability domain for the BP of XOR nodes. The factor graph of Karatsuba in Liboqs is shown in Figure 3b. By performing one BP on the Karatsuba factor graph, we can recover two 4-bit subsets

of the secret key, $t_j[k]$ and $t_{j+2}[k]$. All the bits of the secret key y can be recovered by serially executing BP.

5 Single-trace Key Recovery under Valid Ciphertexts

Most recent works exploit the `expand_and_sum` leakage $c' = v - u \cdot y$ to recover the secret key [DGN25, PRJ+25, MNMD25]. The leakage of recursive Karatsuba is exploited by SASCA during $u \cdot y$ to obtain the probabilities of y. These two leakage points can be measured by an adversary under valid ciphertexts. This section discusses how to combine the two leakages to improve the efficiency of a single-trace attack.

It is necessary to find a method to jointly update the side-channel information of c' and y. In the formula $c' = v - u \cdot y$, u is dense, so it is expensive and infeasible to construct the corresponding factor graph to propagate the probability of c' and y. According to the formula $c' = m\mathbf{G} + e + r_2 \cdot x + r_1 \cdot y$, we can construct a factor graph to perform the BP since the variables r_1, r_2 are both sparse. Dong *et al.* [DGN25] perform single-trace HQC key recovery by parity-check (MDPC) decoding. It applies BP on a joint factor graph over the secret components x and y, using soft information derived solely from ciphertext c' to provide a reliability ranking prior to information set decoding.

In this factor graph, updating the unknown information of x and y at the same time will cause the information of x and y to interfere with each other. In this scenario, side-channel information is available for the component y while x remains constrained primarily by its sparsity prior. The iterative decoding constitutes a fundamental bottleneck of joint BP decoding under this unequal a priori information [Bri00, Hag04]. The information of y will be influenced by the uncertainty of x through shared parity checks.

We present a Turbo-like solver framework to explicitly address the unequal side-channel information across variables. The joint BP problem is decomposed into two coupled soft-in/soft-out (SISO) subproblems, each associated with a single secret component. These subproblems interact in an iterative manner through decision-directed cancellation, following the principles of turbo and decision-directed inference [DJB+95, TB02, TKS02]. The side-channel information of ciphertext c' may contain incorrect information. Prior work on iterative decoding has shown that unreliable information can adversely affect convergence behaviour [Bri00, Hag04]. Motivated by this insight, we introduce a confidence fraction mechanism to activate the subset of high-confidence parity-check constraints during belief propagation. This selective activation limits the influence of error bits and improves the robustness of the iterative solve process.

Let $b = c' + m\mathbf{G} + e$ denote as the vector obtained after truncation by the inner Reed–Muller decoder. Since $m\mathbf{G} + e$ is known, the associated soft information can be derived from the ciphertext c' side-channel information. We denote by $L_b \in \mathbb{R}^{n_1 n_2}$ the LLR vector associated with the observation b, derived from the ciphertext side-channel information.

Each iteration of the Turbo-like solver comprises Decoder 1 for updating the reliability of x given an estimate of y, and Decoder 2 for updating the reliability of y given an estimate of x. At the beginning of iteration t, Decoder 1 receives as input the prior LLRs of y, given by $L_y^{\text{prior}} = L_y^{\text{sca}} + L_w$, where L_y^{sca} denotes the side-channel information on y, and L_w encodes the sparsity prior corresponding to $\text{wt}(y) = \omega$. These prior LLRs are kept fixed across iterations.

An estimate $\hat{y}$ is obtained by projecting the current posterior LLRs onto the set of binary vectors with Hamming weight ω

$$\hat{y} = \text{Proj}_\omega\left(L_y^{\text{post}}\right). \tag{30}$$

This estimate is used exclusively for decision-directed interference cancellation and is not

Algorithm 3 Turbo-Like Solver

Require: $r_1, r_2, m\mathbf{G}, e$; LLRs L_c^{sca}; LLRs L_y^{sca}; Hamming weight ω; turbo iterations T; ciphertext confidence fraction α_c.
Ensure: $(L_x^{\text{post}}, L_y^{\text{post}})$
$c^{\text{sca}} \leftarrow \text{hd}(L_c^{\text{sca}})$ ▷ hard decision from SCA
$b \leftarrow c^{\text{sca}} + m\mathbf{G} + e$
$L_b(j) \leftarrow (-1)^{(m\mathbf{G}+e)_j} L_c^{\text{sca}}(j)$ **for all** j
Select index set $\mathcal{I} \subseteq \{1, \ldots, n_1 n_2\}$ consisting of the $\alpha_c \cdot n_1 n_2$ largest $|L_c^{\text{sca}}(j)|$
Initialize sparsity prior $L_w \leftarrow \log\frac{1-\omega/n}{\omega/n}$
Set $L_x^{\text{prior}} \leftarrow L_w$, $L_y^{\text{prior}} \leftarrow L_y^{\text{sca}} + L_w$, $L_x^{\text{post}} \leftarrow L_x^{\text{prior}}$, $L_y^{\text{post}} \leftarrow L_y^{\text{prior}}$
for $t = 1$ **to** T **do**
    Decoder 1:
    $\hat{y} \leftarrow \text{Proj}_\omega(L_y^{\text{post}})$ ▷ weight-ω projection
    $b_x \leftarrow b + (r_1 \cdot \hat{y})$
    Run BP on $r_2 \cdot x = b_x$ using constraints indexed by $\mathcal{I}$ and prior L_x^{prior}
    $L_x^{\text{post}} \leftarrow$ BP output
    Decoder 2:
    $\hat{x} \leftarrow \text{Proj}_\omega(L_x^{\text{post}})$ ▷ weight-ω projection
    $b_y \leftarrow b + (r_2 \cdot \hat{x})$
    Run BP on $r_1 \cdot y = b_y$ using constraints indexed by $\mathcal{I}$ and prior L_y^{prior}
    $L_y^{\text{post}} \leftarrow$ BP output
end for

interpreted as a final decision. Using $\hat{y}$, Decoder 1 forms a reduced observation

$$b_x = b + r_1 \cdot \hat{y}. \tag{31}$$

Belief propagation is then applied to the equation $r_2 \cdot x = b_x$ using only the parity-check constraints indexed by $\mathcal{I}$, where $\mathcal{I}$ denotes the set of ciphertext c' positions selected according to the reliability of the corresponding side-channel LLRs. Constraints outside $\mathcal{I}$ do not contribute channel information, which reduces the impact of unreliable side-channel information obtained from the ciphertext c'. The BP decoder outputs posterior LLRs L_x^{post}, which are subsequently used to form a weight-ω estimate

$$\hat{x} = \text{Proj}_\omega\big(L_x^{\text{post}}\big) \tag{32}$$

for use in the subsequent stage.

Decoder 2 operates symmetrically to Decoder 1. It receives the fixed *a priori* LLRs of x, given by $L_x^{\text{prior}} = L_w$, and uses the estimate $\hat{x}$ to cancel the contribution of x from the observation

$$b_y = b + r_2 \cdot \hat{x}. \tag{33}$$

Algorithm 3 summarizes the proposed turbo-like solver procedure. After a small number of iterations, the final LLRs $(L_x^{\text{post}}, L_y^{\text{post}})$ provide highly informative reliability rankings, which are exploited by a soft-information ISD algorithm to recover the exact sparse supports of x and y.

6 Applications and Evaluations

In this section, we perform a single-trace attack on HQC. We conduct SASCA on both simulated and practical traces. The attack success rate and execution time are used to evaluate the effectiveness of our methods.

6.1 Evaluation on Simulated Data

We evaluate our method using simulated side-channel traces under different noise levels. The target is the C implementation of HQC-128 submitted to NIST on August 22, 2025. The experiments were conducted on an Intel i7 8750H (2.2 GHz) processor. The simulated traces are generated using a Hamming weight (HW) model and additive Gaussian noise. The simulated traces are represented as $L = \mathrm{HW}(v) + \mathcal{N}(0, \sigma)$, where $\mathcal{N}(0, \sigma)$ denotes a Gaussian distribution with mean 0 and standard deviation σ.

6.1.1 SASCA

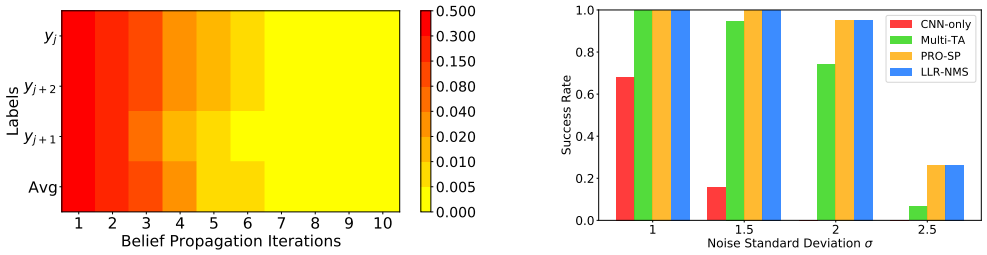
For each noise level, we independently generate 200,000 simulated traces for two leakage points of Karatsuba, y and r (Line 7 of Algorithm 2 and Lines 9 and 10 of Algorithm 2), using Hamming weight and Gaussian noise, respectively. These traces are used to train separate CNN models to classify them, where each trace consists of 50 points and corresponds to one of the labels 0 or 1. For each CNN, 160,000 of the simulated traces are used as the training set and 40,000 as the test set. We refer to the CNN architecture proposed in [ZBHV20] for our work. The CNN architecture and hyperparameters are shown in Table 2. We use the well-trained CNNs to classify the simulated traces from a single execution of Karatsuba. Since the output layer of the CNN uses a softmax activation, a probability distribution of the classification results can be obtained. These probability distributions are used as the prior probability distributions in the factor graph for BP.

Table 2: Overview of the CNN architecture

Layer	Hyperparameters	Dimensions
Input	input_size = 50×1	50×1
Conv1D	filter_num = 8, kernel_size = 1, activation = SELU	50×8
BatchNorm	-	50×8
MaxPooling1D	pool_size = 2, stride = 2	25×8
Flatten	-	200
Dense	units = 50, activation = SELU	50
Dense	units = 10, activation = SELU	10
Output	output_size = 2, activation = Softmax	2

We perform 10 iterations of our LLR-BP based on the normalized min-sum algorithm (LLR-NMS) and use entropy to assess the uncertainty of the probability distributions. Figure 4a visualizes the change in the entropy of the probability distributions of y_j, y_{j+1}, and y_{j+2} after each LLR-NMS iteration at a noise level of $\sigma = 3$. The redder areas indicate higher entropy values, representing greater uncertainty, while the yellower areas indicate lower entropy values. It can be observed that in the initial probability distribution, the entropy is high, resulting in a low attack success rate. As the number of iterations increases, the entropy of the probability distribution gradually decreases. After more than 8 rounds of iterations, the entropy of the probability distribution approaches 0 and no longer updates, indicating that BP has converged.

Subsequently, we comprehensively evaluate the performance of the proposed SASCA in single-trace attack under various noise levels using success rate. In our experiments, single-trace attacks are executed on 10,000 sets of randomly generated HQC full secret keys. The success rate is defined as the proportion of the 10,000 single-trace attacks in which the full secret key y is completely recovered without error. This definition of the success rate is motivated by the requirement of ephemeral key exchange in protocols like TLS 1.3, where a single failed bit recovery would compromise the entire session key [SFG20]. Such an evaluation metric measures the efficiency of single-trace attacks and has been adopted in many works [RRCB20, GMGL24]. Furthermore, given the sparsity of HQC's secret



(a) The changes of entropy during LLR-NMS.

(b) The success rate of SASCAs compared with the traditional attacks.

Figure 4: Convergence and success rate of our SASCAs

key y, which contains only 66 '1's out of 17,669 bits, a metric based on per-bit accuracy would be inflated for all methods, as even an all-zero guess yields a high proportion of correct bits. Such inflation reduces the discriminative capacity between different methods. In contrast, the success rate definition adopted in our work provides a more stringent evaluation metric and better highlights the differences between methods, especially under low-noise conditions. We evaluate the success rate of our LLR-NMS method at noise levels of $\sigma = 1, 1.5, 2, 2.5$, and compare with the CNN-only method, the Multivariate TA [CK13], and the probability-domain BP based on the sum-product algorithm (PRO-SP). The comparison results are illustrated in Figure 4b. Specifically, the CNN-only method trains on the concatenated traces from all leakage points (Lines 7, 9, and 10 of Algorithm 2), and the Multivariate TA similarly uses the concatenated traces to build a joint template. Each concatenated trace consists of one trace of y followed by 9 traces of r, and is labeled as either 0 or 1. The CNN-only uses 16,000 concatenated traces for training and 4,000 for testing, while the Multivariate TA constructs templates using 10,000 concatenated traces for each class. In both cases, the total number of traces consumed is identical to that used in SASCA.

The experimental results presented in Figure 4b demonstrate that the CNN-only method focuses on the strongest leakage point in the trace, failing to effectively aggregate the information across multiple leakage points. While the Multivariate TA effectively captures all leakage points within the Schoolbook implementation, thereby significantly enhancing the success rates, it fails to model the inherent algebraic relationships within the Karatsuba algorithm. In contrast, SASCA can further leverage the additive constraints between any two bits of y introduced by the Karatsuba decomposition, thereby maximizing the utilization of all available side-channel leakage. Consequently, SASCA achieves superior attack success rates compared to the Multivariate TA, particularly in high-noise scenarios, even under the strict success rate definition adopted in our attack where a few incorrect bits lead to a recovery failure. Furthermore, our LLR-NMS achieves the same success rates as the PRO-SP, indicating that LLR-NMS converges to identical hard decisions as PRO-SP.

However, our SASCA demonstrates a significant advantage in terms of time efficiency. Table 3 compares the PRO-SP, the LLR-BP based on the sum-product algorithm (LLR-SP), and our LLR-NMS. The experimental results show that our SASCA requires only 335.88 seconds to perform a complete BP. In contrast, the PRO-SP takes 4476.64 seconds, and the LLR-SP takes 412.67 seconds. Compared to the PRO-SP, our method achieves an approximately 13.3 times efficiency improvement without sacrificing accuracy, significantly reducing the time cost for the attack. After modifying the sum-product algorithm to the normalized min-sum algorithm, our SASCA still reduces the execution time by 18.6%.

Table 3: Comparison of execution times (seconds) for different BP methods.

Method	**PRO-SP**	**LLR-SP**	**LLR-NMS**
Execution Time	4476.64	412.67 (×10.8)	335.88 (×13.3)

Table 4: Comparison of BP results for different methods under various noise levels.

Number of errors and complexity (N_x,N_y,C_{trial})					
σ	N_y	$N_{c'}$	MDPC [DGN25]	Directed MDPC	Ours
2	0	718	(16,0,2^{43})	(16,0,2^{43})	(0,0,2^{43})
3	2	2205	(48,4,2^{43})	(30,2,2^{43})	(0,0,2^{43})
4	4	3396	(90,8,2^{45})	(84,4,2^{43})	(0,0,2^{43})
5	24	4337	(122,50,2^{148})	(114,24,2^{78})	(0,0,2^{43})
6	48	4975	(128,66,2^{194})	(126,48,2^{148})	(0,0,2^{43})
Performance					
			MDPC [DGN25]	Directed MDPC	Ours
Execution Time			5067.16 s	5044.78 s	10.25 s

6.1.2 Single-trace Key Recovery under Valid Ciphertexts

We evaluate the efficiency of key recovery by exploiting the side-channel information of c' and y. The simulated leakage traces for c' and $u \cdot y$ are generated using noise levels of $\sigma = 2, 3, 4, 5, 6$. We employ the well-trained CNN and the LLR-NMS introduced in Section 6.1.1 to execute the SASCA, thereby obtaining the probability distributions of y from the leakages of Karatsuba. Concurrently, the probability distributions associated with c' are obtained through the training and matching of an additional CNN, which shares the identical architecture and hyperparameter configurations as detailed in Table 2.

The computational cost per trial, denoted C_{trial}, is mainly incurred by three components: the Gaussian elimination, the generation of partial-syndrome lists, and the subsequent collision search. It can be approximated as

$$C_{\text{trial}} \approx C_{\text{Gauss}} + 2\binom{(n+\ell)/2}{w_0} \cdot w_0 \cdot \ell \tag{34}$$

The term C_{Gauss} reflects the complexity of the Gaussian elimination performed prior to the enumeration phase. The subsequent search dominates the remaining cost, as it requires the generation of two lists of size $\binom{(n+\ell)/2}{w_0}$ together with the evaluation of partial syndromes, whose complexity scales linearly with $w_0\ell$. For the HQC-128 parameter set with $\ell = 51$, we fix $w_0 = 3$, since the meet-in-the-middle step involves lists of size $\binom{(n+\ell)/2}{w_0}$, which grow combinatorially with w_0. For the HQC-128 parameter set with $\ell = 51$, we fix $w_0 = 3$, since the meet-in-the-middle step involves lists of size $\binom{(n+\ell)/2}{w_0}$, which grow combinatorially with w_0.

The MDPC-based method of Dong *et al.* [DGN25] relies solely on side-channel information of c'. We apply this MDPC method in the experiments by additionally integrating the side-channel information on y into the same factor graph. Belief propagation jointly updates the probabilities of x and y, while the directed variant fixes y first to update x. We run the proposed Turbo-like solver for 10 iterations of BP, which takes approximately 10 seconds, whereas the MDPC method and the directed variant take more than one hour to output the probabilities of x and y.

Table 4 summarizes the post-processing results under different error levels induced by the simulated side-channel noise. For a given noise level σ, the values N_y corresponds to the numbers of bit errors in the secret component y after SASCAs. $N_{c'}$ denotes the numbers of bit errors of c' for attacking the `expand_and_sum`. For each method, the tuple $(N_x, N_y, C_{\text{trial}})$ reports the number of residual bit errors in x and y and the ISD trial costs

Table 5: Evaluation under different confidence thresholds.

σ	N_y	$N_{c'}$	Ours $(N_x, N_y, C_{\text{trial}})$				
			$\alpha_c = 0.1$	$\alpha_c = 0.2$	$\alpha_c = 0.3$	$\alpha_c = 0.4$	$\alpha_c = 0.5$
2	0	718	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})
3	2	2205	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})
4	4	3396	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})
5	24	4337	(10,20,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})
6	48	4975	(48,102,2^{194})	(38,56,2^{43})	(0,0,2^{43})	(0,0,2^{43})	(0,0,2^{43})

C_{trial}. It should be noted that, the results in Table 4 are obtained without actual ISD execution. Instead, we count the residual errors in x and y after the Turbo-like solver and after the MDPC decoding, respectively. The ISD trial cost C_{trial} is estimated via Eq. (34), which depends on the number of residual errors in x and y. This analytical approach follows the standard ISD evaluation framework adopted in recent works [DGN25, BMG+25]. As shown in Table 4, we observe that the proposed method consistently eliminates all residual errors after BP refinement, keeping the ISD post-processing cost at the baseline level $C_{\text{trial}} = 2^{43}$, as obtained by evaluating Eq. (34) with $w_0 = 0$. The existing methods exhibit the rapid accumulation of residual errors as the noise level increases, resulting in an explosive growth of the ISD trial cost from 2^{45} up to 2^{194} or 2^{43} up to 2^{148}.

The index set $\mathcal{I}$ is constructed by retaining a fixed fraction of ciphertext positions according to their side-channel reliability. Specifically, we select

$$|\mathcal{I}| = \lfloor \alpha_c \cdot n_1 n_2 \rfloor \tag{35}$$

ciphertext positions with the largest absolute LLR values and perform BP on the restricted equation $r_2 \cdot x = b_x$ using only the corresponding constraints. The post-processing results obtained with the proposed method under varying values of α_c are shown in Table 5. The experimental results show that there is no need to choose strict parameter values, and the parameters have a loose range in our method.

6.1.3 Compared with Related Works

Several existing works similarly employ BP to execute single-trace attacks on HQC. To demonstrate the superiority of our proposed method, this section provides a comprehensive comparison with related works across multiple dimensions, including the attack target, success rate, and execution time. The compared methods include those from [GMGL24], [BMG+25], and [DGN25]. The comparative experiments are conducted on simulated traces under varying noise levels of $\sigma = 2, 3, 4, 5, 6$, with the results summarized in Table 6. It is worth noting that the reported attack results for all methods solely reflect the success rates prior to ISD, excluding the ISD post-processing phase. The execution time excludes the overhead associated with template matching and ISD. For our proposed approach, it corresponds to the total execution time of the BP iterations and the Turbo-like solver.

[DGN25] is the most closely related study to our threat model. Their approach recovers the static secret key (x, y) using a single valid ciphertext, by exploiting the side-channel leakage of the `expand_and_sum` operation combined with MDPC decoding. As demonstrated in Section 6.1.2, [DGN25] exhibits poor noise robustness because its BP relies on the prior probability distribution of c'. Consequently, the number of error bits increases as the noise level increases, leading to an explosive increase in the complexity of the subsequent ISD. In contrast, our method eliminates errors after using the Turbo-like solver under the same high-noise conditions, maintaining the subsequent ISD recovery cost at the baseline level. Furthermore, the joint update mechanism employed by [DGN25] requires nearly 1.5 hours of BP iterations to output probabilities. Our work utilizes the

Table 6: Comparison of different methods on single-trace attacks on HQC-128.

Feature	Ours	[DGN25]	[BMG+25]	[GMGL24]
Target	Karatsuba	`expand_and_sum`	FHT	`gf_mul`
Recovered Key	(x, y)	(x, y)	y^*	m
Ciphertext	Valid	Valid	Invalid	Valid
Time	346.13 s	5067.16 s	7217.80 s	-
		Number of Errors		
$\boldsymbol{\sigma = 2}$	(0, 0)	(16, 0)	0	-
$\boldsymbol{\sigma = 3}$	(0, 0)	(48, 4)	0	-
$\boldsymbol{\sigma = 4}$	(0, 0)	(90, 8)	2	-
$\boldsymbol{\sigma = 5}$	(0, 0)	(122, 50)	16	-
$\boldsymbol{\sigma = 6}$	(0, 0)	(128, 66)	12	-

efficient LLR-NMS algorithm, and the Turbo-like solver achieves acceleration by decoupling the updates of x and y, completing the entire attack flow in only 346.13 seconds.

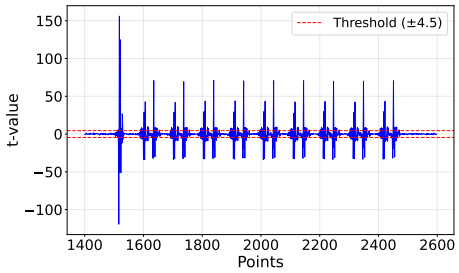
[BMG+25] utilizes SASCA exploiting the leakage of the FHT within the RM decoder to recover the secret key y. However, their attack relies on chosen invalid ciphertexts to force the RM decoder to process y. Such an assumption is resisted by invalid ciphertext detection mechanisms. In contrast, our proposed attack relies solely on a single valid ciphertext, offering higher practical viability. More importantly, while [BMG+25] targets the input of the FHT, it is the sum of multiple secret key bits $y^* = \sum_{i=1}^{mul} y^{(i)}$ rather than the secret key. This accumulation introduces positional ambiguity, making it impossible to pinpoint the exact locations of the non-zero bits. As a result, [BMG+25] relies on multiple iterations of ISD as a post-processing step to resolve this positional ambiguity. The attack results for this method presented in Table 6 denote the number of errors in the recovered y^*. It is worth noting that BP convergence is adversely affected when the FHT input y^* contains at least one value of 2, leading to a degradation in the success rate. This explains why the number of errors at $\sigma = 5$ is higher than that at $\sigma = 6$. Furthermore, due to the massive number of variable nodes involved in the 7-layer butterfly operation of the FHT, its BP execution is also time-consuming.

[GMGL24] constructs a factor graph exploiting the side-channel leakage of the `gf_mul` function within the RS decoder of HQC. However, their objective is to extract the encapsulated message m to subsequently derive the ephemeral shared key. In contrast, our method focuses on recovering the long-term static secret key y. Due to the fundamental difference in the targets of the two attacks, they are not comparable.

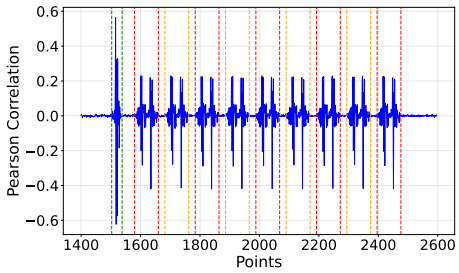
In summary, the proposed method not only extends the attack to the previously unexplored Karatsuba multiplication, but also achieves a breakthrough in both attack success rate and execution efficiency under the assumption of valid ciphertexts.

6.2 Evaluation on Real Data

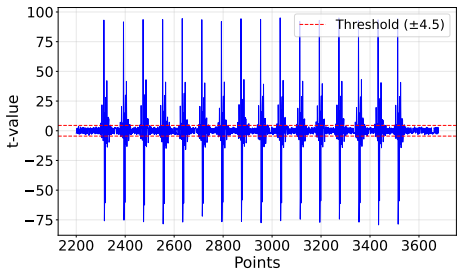
The target device for the actual attack is an ARM Cortex-M4 microcontroller STM32F407 running at 168 MHz. STM32CubeIDE is used to compile and program the device. We run both the HQC reference implementation and the Liboqs implementation on this device, and record the microcontroller's electromagnetic (EM) radiation traces using a PicoScope 3205D digital oscilloscope and a Langer RF2 near-field probe. All measurements are taken with a sampling rate of 100 MSa/s. The choice of EM is due to the absence of a direct power measurement interface on the STM32F407. Moreover, we find that capturing EM using a near-field probe on the chip's surface is effective. Placing a circular loop probe flat against the top of the package yields distinct side-channel leakage, bypassing the need for precise physical probe positioning.



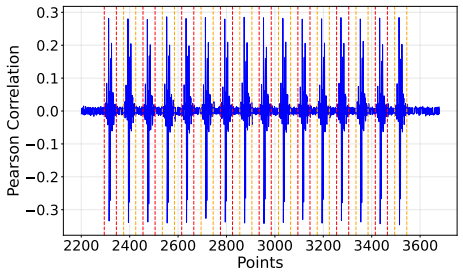
(a) TVLA for $y[1]$ in the reference implementation with the ±4.5 threshold.



(b) Pearson correlation for $y[1]$ in the reference implementation.



(c) TVLA for $t[1]$ in the Liboqs implementation with the ±4.5 threshold.

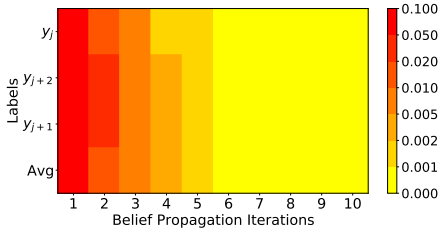


(d) Pearson correlation for $t[1]$ in the Liboqs implementation.

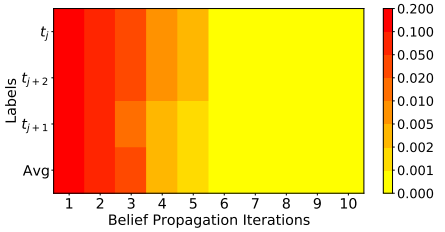
Figure 5: Leakage detection and PoI selection for the reference and Liboqs implementations using TVLA and Pearson correlation.

For the reference implementation, we execute the Karatsuba multiplication and collect its EM traces. Each raw EM trace consists of 1,800,000 points and covers the three Schoolbook multiplications performed during the 4-th recursion round of Karatsuba. Figure 5a shows the TVLA [GGJR+11] computed at the trace positions corresponding to the secret key bit $y[1]$, where the x-axis indicates the sample index in the raw trace. The t-values far exceed the ±4.5 threshold, confirming the presence of strong leakage. Pearson correlation is used to identify the PoIs [DS16]. Figure 5b shows the Pearson correlation between the secret key bit $y[1]$ and its corresponding trace positions, where the PoIs are selected as the points corresponding to the peaks of the correlation curve, as highlighted by the dashed vertical lines in Figure 5b. The first peak corresponds to the PoI of $y[1]$, while the remaining lower peaks correspond to the PoIs of $r[0]$ through $r[8]$. After PoI extraction, we obtain 100,000 traces of y and $100,000 \cdot n$ traces of r. Each trace of y consists of 35 points, and each trace of r consists of 80 points. To perform SASCA, we use 100,000 traces for each of y and r to train the CNN, with 75,000 traces used for training and 25,000 for testing. The CNN architecture is similar to the settings in Table 2. To perform the Multivariate TA, we select 20,000 traces of y, and concatenate each trace of y with its corresponding 9 traces of r in the original computational order of the Schoolbook multiplication, forming 20,000 concatenated traces for template building. This preserves the algorithmic structure of the target operation while reducing noise and improving template quality. A total of 200,000 extracted traces (equivalent to 20,000 concatenated traces) are used in the Multivariate TA template building, which is the same as the number used in SASCA. Each concatenated trace is labeled as either 0 or 1, with 10,000 concatenated traces for label 0 and 10,000 for label 1. The experimental results demonstrate that while the Multivariate TA achieves an averaged error rate of 0.012% on each bit, its overall success rate remains at 12.6%. The error rate indicates the

proportion of incorrect key bits relative to the complete key. Then we perform the SASCA attack on the same dataset. After 10 rounds of LLR-NMS BP updates, the success rate for y reaches 100%. We visualize the change in entropy of the probability distribution for y_j, y_{j+1}, y_{j+2} after each round of BP iteration in Figure 6a. Due to the high quality of the prior probability distribution, the initial entropy values in the practical traces are relatively low and convergence is achieved after 6 rounds of BP iterations.



(a) The changes of entropy in the practical traces of the reference implementation.



(b) The changes of entropy in the practical traces of the Liboqs implementation.

Figure 6: The changes of entropy during belief propagation in practical traces.

For the Liboqs implementation, we similarly collect EM traces of the Karatsuba multiplication. Each raw trace consists of 80,000 points and contains three base_mul operations. A similar leakage analysis is performed on these traces. Specifically, we compute the TVLA and Pearson correlation on the raw traces for the second 4-bit subset of the secret key y, denoted as $t[1]$. The TVLA and Pearson correlation results are shown in Figure 5c and Figure 5d, where the x-axis indicates the sample index in the raw trace. The TVLA exhibits t-values well above the ± 4.5 threshold, confirming the presence of strong leakage. The peaks of the Pearson correlation traces are then used to select the PoIs. These peaks correspond to the leakage positions of $b[0]$ through $b[15]$. After PoI extraction, we obtain 100,000 traces for the leakage point b of Karatsuba, with 75,000 traces used for training and 25,000 for testing. Each trace consists of 50 points. We apply SASCA to recover each 4-bit subset t of the secret key y. In our experiments, we perform the attack on 1,000 traces and calculate the attack success rate. Ultimately, the success rate for y reaches 100%. We visualize the change in entropy of the probability distribution for t_j, t_{j+1}, t_{j+2} after each round of BP iteration in Figure 6b. Similarly, the BP also converges after 6 rounds of iterations.

6.3 Attacks on Masking

Codeword masking [MSS13] is widely used to defend against side-channel attacks during decoding. Previous attacks mainly targeted the HQC decoder, leading current masking countermeasures for HQC to focus primarily on codeword masking [MNMD25, HWZY25]. The core idea is to generate a random message mask m' and compute $v' = v + m'\mathbf{G}$ before decoding, where $m'\mathbf{G}$ is the codeword corresponding to m'. During decoding, the algorithm processes the sum of the original ciphertext and the codeword mask, resulting in $m + m' = D_{\text{decode}}(v - u \cdot y + m'\mathbf{G})$. Finally, the mask m' is removed via a simple subtraction to complete the unmasking process. However, our proposed SASCA targeting the Karatsuba multiplication can directly recover the secret key y from the computation of $u \cdot y$. Therefore, solely masking the message m is insufficient to defeat our attack. Furthermore, even when codeword masking is deployed, the adversary can exploit side-channel leakages during the Reed-Solomon encoding phase (i.e., the computation of $m'\mathbf{G}$). By combining template classification with Gaussian elimination, the random mask m' can be extracted in advance, thereby breaking the codeword protection and recovering the input of `expand_and_sum` [DGN25].

In addition to codeword masking, a masked sampler was designed in [SVP+24]. The masked sampler directly generates Boolean shares using a True Random Number Generator (TRNG), and securely computes the shares of y entirely within the masked domain via a masked Barrett reduction and a secure shuffling algorithm. As noted by [GSA+25] in their discussion of comprehensive masking strategies for HQC, y is a sensitive variable that must be protected during the polynomial multiplication $u \cdot y$. To evaluate the effectiveness of our SASCA against masked Karatsuba implementations, we conduct experiments on the real device. The HQC masked sampler generates first-order Boolean shares y_1 and y_2 of the secret key y, then consecutively executes the Karatsuba multiplications $u \cdot y_1$ and $u \cdot y_2$. We perform the attack on the ARM Cortex-M4 microcontroller STM32F407 to collect side-channel traces. Since both multiplication branches are performed sequentially within a single decapsulation, we can construct independent factor graphs for each Karatsuba and execute a single-trace SASCA on the same trace. Experimental results demonstrate that, after BP updates, both y_1 and y_2 are recovered with a 100% success rate. The complete secret key y is recovered by combining the shares. These results confirm that the proposed SASCA remains highly effective even in masked scenarios.

7 Conclusion

In this work, we present a comprehensive side-channel analysis of the latest HQC reference implementation and its Liboqs implementation, targeting the iterative Karatsuba polynomial multiplication. By constructing and optimizing factor graphs tailored to the specific binary values and sparsity of HQC, we demonstrate that the proposed SASCAs are a practical threat to this scheme. Our work introduces a novel joint recovery framework that bridges the gap between polynomial multiplication and decoding. By combining the leakage from the Karatsuba algorithm with the critical `expand_and_sum` operation, and applying our proposed bi-directional solver, we successfully perform a single-trace attack using only legitimate ciphertexts. The results underscore that securing the decoding step alone is insufficient. Therefore, our study suggests that future developments in HQC implementation should focus on deriving efficient countermeasures to prevent such SASCA methodologies.

Acknowledgements

This work was supported in part by National Natural Science Foundation of China under Grant (NO.62472259, NO.62572285), the Program of Taishan Young Scholars of Shandong Province (NO.tsqn202408026), Shandong Provincial Natural Science Foundation (ZR2024QF098).

References

[BCLVV18] Daniel J Bernstein, Chitchanok Chuengsatiansup, Tanja Lange, and Christine Van Vredendaal. Ntru prime: Reducing attack surface at low cost. In *Selected Areas in Cryptography – SAC 2017*, pages 235–260, Cham, 2018. Springer International Publishing.

[BDK+18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, London, UK, 2018.

[BMG+25] Chloé Baïsse, Antoine Moran, Guillaume Goy, Julien Maillard, Nicolas Aragon, Philippe Gaborit, Maxime Lecomte, and Antoine Loiseau. Secret and shared keys recovery on hamming quasi-cyclic with sasca. *Designs, Codes and Cryptography*, 93(6):2137–2157, 2025.

[Bri00] S ten Brink. Designing iterative decoding schemes with the extrinsic information transfer chart. *AEU Intl. J. Electr. and Commun.*, 54(6):389–398, 2000.

[CK13] Omar Choudary and Markus G Kuhn. Efficient template attacks. In *International Conference on Smart Card Research and Advanced Applications*, pages 253–270. Springer, 2013.

[DG25] Haiyue Dong and Qian Guo. Ot-pca: New key-recovery plaintext-checking oracle based side-channel attacks on hqc with offline templates. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):251–274, 2025.

[DGN25] Haiyue Dong, Qian Guo, and Denis Nabokov. Single-trace key recovery attacks on hqc using valid and invalid ciphertexts. *Cryptology ePrint Archive*, 2025.

[DJB+95] Catherine Douillard, Michel Jézéquel, Claude Berrou, Département Electronique, Annie Picart, Pierre Didier, and Alain Glavieux. Iterative correction of intersymbol interference: turbo-equalization. *European transactions on telecommunications*, 6(5):507–511, 1995.

[DKL+18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.

[DKSRV18] Jan-Pieter D'Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *International Conference on Cryptology in Africa*, pages 282–305. Springer, Cham, 2018.

[DS16] François Durvaux and François-Xavier Standaert. From improved leakage detection to the detection of points of interests in leakage traces. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 240–262. Springer, 2016.

[Gal03] Robert Gallager. Low-density parity-check codes. *IRE Transactions on information theory*, 8(1):21–28, 2003.

[GAMA+25] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.

[GGJR+11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. A testing methodology for side-channel resistance validation. In *NIST non-invasive attack testing workshop*, volume 7, pages 115–136, 2011.

[GJ20] Qian Guo and Thomas Johansson. A new decryption failure attack against hqc. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 353–382. Springer, 2020.

[GMGL24] Guillaume Goy, Julien Maillard, Philippe Gaborit, and Antoine Loiseau. Single trace hqc shared key recovery with sasca. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):64–87, 2024.

[GSA+25] Guillaume Goy, Maxime Spyropoulos, Nicolas Aragon, Philippe Gaborit, Renaud Pacalet, Fabrice Perion, Laurent Sauvage, and David Vigilant. Side-channel sensitivity analysis on hqc: Towards a fully masked implementation. *Cryptology ePrint Archive*, 2025.

[Hag04] Joachim Hagenauer. The exit chart-introduction to extrinsic information transfer in iterative processing. In *2004 12th European Signal Processing Conference*, pages 1541–1548. IEEE, 2004.

[HHP+21] Mike Hamburg, Julius Hermelink, Robert Primas, Simona Samardjiska, Thomas Schamberger, Silvan Streit, Emanuele Strieder, and Christine van Vredendaal. Chosen ciphertext k-trace attacks on masked cca2 secure kyber. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):88–113, Aug. 2021.

[HWZY25] Zhuo Huang, Weijia Wang, Xiaogang Zhou, and Yu Yu. Dpa-style attacks on hqc. *Cryptology ePrint Archive*, 2025.

[KO62] Anatolii Alekseevich Karatsuba and Yu P Ofman. Multiplication of many-digital numbers by automatic computers. In *Doklady Akademii Nauk*, volume 145, pages 293–294. Russian Academy of Sciences, 1962.

[LZH+22] Yanbin Li, Jiajie Zhu, Yuxin Huang, Zhe Liu, and Ming Tang. Single-trace side-channel attacks on the toom-cook: The case study of saber. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 285–310, 2022.

[LZZ+25] Zhenzhi Lai, Ruiyi Zhang, Zhiyuan Zhang, Julius Hermelink, Michael Schwarz, Van-Thuan Pham, and Udaya Parampalli. You only decapsulate once: Ciphertext-independent single-trace passive side-channel attacks on hqc. *Cryptology ePrint Archive*, 2025.

[MAB+18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loıc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.

[MNMD25] Nathan Maillet, Cyrius Nugier, Vincent Migliore, and Jean-Christophe Deneuville. Key recovery from side-channel power analysis attacks on non-simd hqc decryption. In *Annual International Cryptology Conference*, pages 70–102. Springer, 2025.

[MSS13] Dominik Merli, Frederic Stumpf, and Georg Sigl. Protecting puf error correction by codeword masking. *Cryptology ePrint Archive*, 2013.

[PP19] Peter Pessl and Robert Primas. More practical single-trace attacks on the number theoretic transform. In *Progress in Cryptology – LATINCRYPT 2019*, pages 130–149, Cham, 2019. Springer, Cham.

[PPM17] Robert Primas, Peter Pessl, and Stefan Mangard. Single-trace side-channel attacks on masked lattice-based encryption. volume 10529, pages 513–533, 2017.

[PRJ+25] Thales B Paiva, Prasanna Ravi, Dirmanto Jap, Shivam Bhasin, Sayan Das, and Anupam Chattopadhyay. Et tu, brute? side-channel assisted chosen ciphertext attacks using valid ciphertexts on hqc kem. In *International Conference on Post-Quantum Cryptography*, pages 294–321. Springer, 2025.

[PT19] Thales Bandiera Paiva and Routo Terada. A timing attack on the hqc encryption scheme. In *International Conference on Selected Areas in Cryptography*, pages 551–573. Springer, 2019.

[RBG+25] Tomáš Rabas, Jiří Buček, Vincent Grosso, Karolína Zenknerová, and Róbert Lórencz. Profiling side-channel attack on hqc polynomial multiplication using machine learning methods. In *International Conference on Constructive Approaches for Security Analysis and Design of Embedded Systems*, pages 580–602. Springer, 2025.

[RCDB24] Prasanna Ravi, Anupam Chattopadhyay, Jan Pieter D'Anvers, and Anubhab Baksi. Side-channel and fault-injection attacks over lattice-based post-quantum schemes (kyber, dilithium): Survey and new results. *ACM Transactions on Embedded Computing Systems*, 23(2):1–54, 2024.

[RRCB20] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on cca-secure lattice-based pke and kems. *IACR transactions on cryptographic hardware and embedded systems*, pages 307–335, 2020.

[SFG20] Douglas Stebila, Scott Fluhrer, and Shay Gueron. Hybrid key exchange in tls 1.3. *IETF draft*, 2020.

[SHR+22] Thomas Schamberger, Lukas Holzbaur, Julian Renner, Antonia Wachter-Zeh, and Georg Sigl. A power side-channel attack on the reed-muller reed-solomon version of the hqc cryptosystem. In *International Conference on Post-Quantum Cryptography*, pages 327–352. Springer, 2022.

[SRSWZ20] Thomas Schamberger, Julian Renner, Georg Sigl, and Antonia Wachter-Zeh. A power side-channel attack on the cca2-secure hqc kem. In *International Conference on Smart Card Research and Advanced Applications*, pages 119–134. Springer, 2020.

[SVP+24] Maxime Spyropoulos, David Vigilant, Fabrice Perion, Renaud Pacalet, and Laurent Sauvage. Masked vector sampling for hqc. *Cryptology ePrint Archive*, 2024.

[TB02] Stephan Ten Brink. Convergence behavior of iteratively decoded parallel concatenated codes. *IEEE transactions on communications*, 49(10):1727–1737, 2002.

[TKS02] Michael Tuchler, Ralf Koetter, and Andrew C Singer. Turbo equalization: Principles and new results. *IEEE transactions on communications*, 50(5):754–767, 2002.

[TO10] Naohiro Tsuji and Tomoaki Ohtsuki. Initial llr setting for belief propagation decoding in relay system. In *2010 IEEE International Conference on Communications*, pages 1–5. IEEE, 2010.

[Too63] Andrei L Toom. The complexity of a scheme of functional elements realizing the multiplication of integers. In *Soviet Mathematics Doklady*, volume 3, pages 714–716, 1963.

[VCGS14] Nicolas Veyrat-Charvillon, Benoît Gérard, and François-Xavier Standaert. Soft analytical side-channel attacks. In *Advances in Cryptology – ASIACRYPT 2014*, pages 282–296. Springer, Berlin Heidelberg, 2014.

[WTBB+19] Guillaume Wafo-Tapa, Slim Bettaieb, Loïc Bidoux, Philippe Gaborit, and Etienne Marcatel. A practicable timing attack against hqc and its countermeasure. *Cryptology ePrint Archive*, 2019.

[XPR+21] Zhuang Xu, Owen Pemberton, Sujoy Sinha Roy, David Oswald, Wang Yao, and Zhiming Zheng. Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of kyber. *IEEE Transactions on Computers*, 71(9):2163–2176, 2021.

[ZBHV20] Gabriel Zaid, Lilian Bossuet, Amaury Habrard, and Alexandre Venelli. Methodology for efficient cnn architectures in profiling attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(1):1–36, 2020.